

Functional Programming in F# from Zero to Hero

Internal Engineering Learning Series

A decorative horizontal bar at the bottom of the slide, consisting of two overlapping blue bars with a gradient from light blue to dark blue.

Session 2

F# LANGUAGE BASICS

A decorative horizontal bar at the bottom of the slide, consisting of three overlapping blue bands of varying shades, creating a gradient effect.

Session 2 Plan

F# language basics

1. Review F# syntax and semantics.
 2. Understand bindings, expressions, indentation, and type inference.
 3. Use F# Interactive for quick feedback.
 4. Whitespace and indentation define structure.
- 
- A decorative horizontal bar at the bottom of the slide, consisting of three overlapping blue rectangles of varying shades, creating a gradient effect.

Functions

```
int Add(int x, int y)
{
    return x + y;
}
```

Func<int,int,int>

↖ ↗
In Out

```
let add x y = x + y
```

int -> int -> int

↖ ↗
In Out

let instead of
let ... in
non-terminating
comprehensions

F# Types of Types

- Tuples
- Record Types
- Discriminated Unions
- Measurements

Tuples

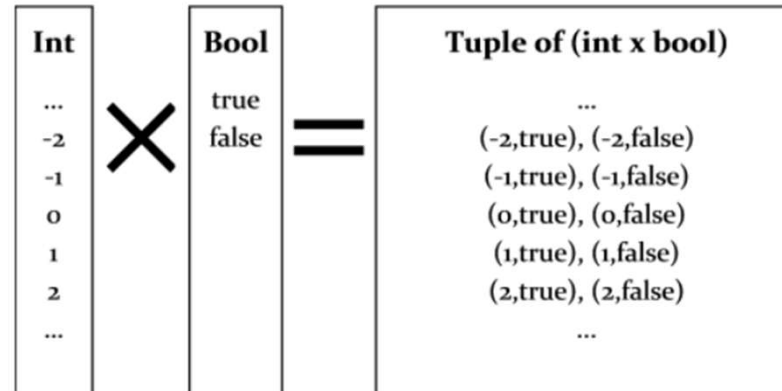
```
let divide dividend divisor =  
  let quotient = dividend / divisor  
  let remainder = dividend % divisor  
  (quotient, remainder)
```

```
let quotient, remainder = divide 10 3
```

Tuples

```
// Construct
let z = 1,true,"hello",3.14

// Deconstruct
let z1,_,z3,z4 = z
```



- No explicit names
- Can have any number of components
- Components can be generic or complex types
- Most often used to return multiple values from private functions

Records

```
// Definition
```

```
type Point =
```

```
{
```

```
  x: float
```

```
  y: float
```

```
}
```

```
static member (-) (a: Point, b: Point) =
```

```
    { x = a.x - b.x; y = a.y-b.y }
```

```
member p.Magnitude
```

```
    with get() =
```

```
        sqrt <| (pown p.x 2) + (pown p.y 2)
```

```
// Construct
```

```
let point = { x = 5. ; y = 0.25 }
```

- Like tuples with labels
- Structural equality by default
- Can have members

```
// Deconstruct
```

```
let { y = deconstructedY } = point
```

```
printfn "%f" deconstructedY
```


Records

```
type DivisionResult = {  
  Quotient: int  
  Remainder: int  
}
```

```
let result = { Quotient = 3; Remainder = 1 }
```

```
let result = { Quotient = 3; Remainder = 1 } : DivisionResult
```

```
let newResult = { Quotient = result.Quotient; Remainder = 0 }
```

```
let newResult = { result with Remainder = 0 }
```

```
let result1 = { Quotient = 3; Remainder = 1 }
```

```
let result2 = { Quotient = 3; Remainder = 1 }
```

```
result1 = result2 // true
```

Structural Equality
Reference Types



“Modifying” an F# Record Type

```
let game1 = {  
    Name = "Halo 5"  
    Developer = "343 Industries"  
    Year = 2015  
    Platforms = [ "Xbox One" ]  
}
```

```
let game2 = { game1 with Platforms = "PC" :: game1.Platforms }
```

Discriminated Unions

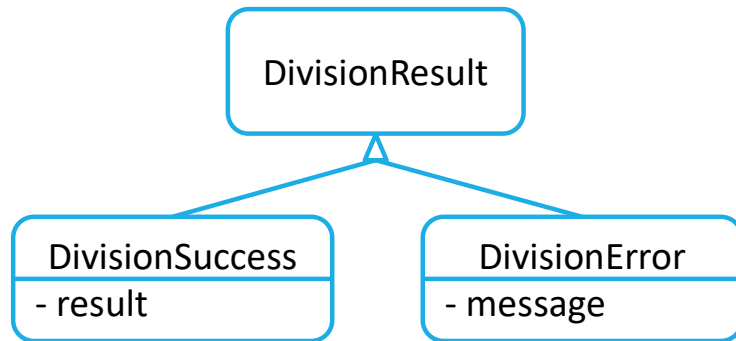
```
type Shape =  
  | Rectangle of width: float * length: float  
  | Circle of radius: float  
  | Prism of width: float * float * height: float
```

```
type Result =  
  | Success  
  | Error of string
```

```
type Tree =  
  | Tip  
  | Node of int*Tree*Tree
```

- Mostly used with pattern matching
- Can be recursive
- Can have field names (tuple labels)
- Empty cases possible
- Option union instead of null

Discriminated Unions



```
type DivisionResult =  
  | DivisionSuccess of result : int  
  | DivisionError of message : string
```

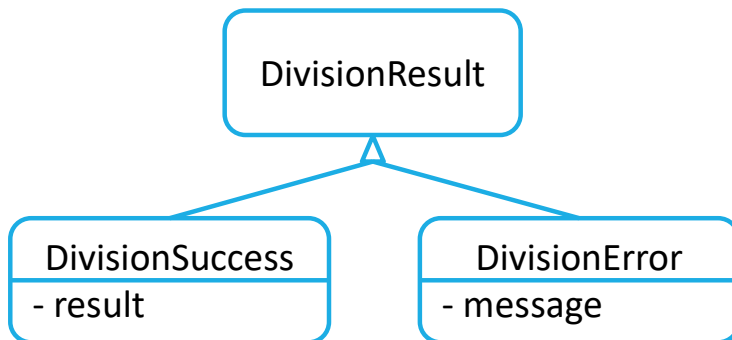
Discriminated Unions

```
let divide x y =  
  match y with  
  | 0 -> DivisionError("Divide by zero")  
  | _ -> DivisionSuccess(x / y)
```

```
let result = divide 4 0  
match result with  
| DivisionSuccess result -> printfn "Result: %i" result  
| DivisionError message -> printfn "Error: %s" message
```

Discriminated Unions

```
type Boolean =  
  | True  
  | False
```



```
type DivisionResult =  
  | DivisionSuccess of result : int  
  | DivisionError of message : string
```

Discriminated Unions

```
let divide x y =  
  match y with  
  | 0 -> DivisionError("Divide by zero")  
  | _ -> DivisionSuccess(x / y)
```

```
let result = divide 4 0  
match result with  
| DivisionSuccess result -> printfn "Result: %i" result  
| DivisionError message -> printfn "Error: %s" message
```

Oh, and Null


```
type Shape =  
    | Circle of float  
    | Square of float  
    | Triangle of float * float  
    | Rectangle of float * float  
  
let GetArea shape =  
    match shape with  
    | Circle(radius) -> 3.14 * radius * radius  
    | Square(width) -> width * width  
    | Triangle(``base``, height) -> ``base`` * height / 2.0  
    | Rectangle(width, height) -> width * height
```



F#

```
type Shape =  
    | Circle of float  
    | Square of float  
    | Triangle of float * float  
    | Rectangle of float * float  
  
let GetArea shape =  
    match shape with  
    | Circle(radius) -> 3.14 * radius * radius  
    | Square(width) -> width * width  
    | Triangle(``base``, height) -> ``base`` * height / 2.0  
    | Rectangle(width, height) -> width * height
```

F# Interactive

>



F#

```
type Shape =  
    | Circle of float  
    | Square of float  
    | Triangle of float * float  
    | Rectangle of float * float  
  
let GetArea shape =  
    match shape with  
    | Circle(radius) -> 3.14 * radius * radius  
    | Square(width) -> width * width  
    | Triangle(``base``, height) -> ``base`` * height / 2.0  
    | Rectangle(width, height) -> width * height
```

```
F# Interactive  
> GetArea null;;
```



F#

```
type Shape =  
    | Circle of float  
    | Square of float  
    | Triangle of float * float  
    | Rectangle of float * float
```

```
let GetArea shape =
```

```
    match shape with
```

**MAKE INVALID STATES
UNREPRESENTABLE**

F# Interactive

```
> GetArea null,
```

error FS0043: The type 'Shape' does not have 'null' as a proper value

```
>
```



Pattern Matching Guards

```
let divide x y =  
  match y with  
  | 0 -> DivisionError("Divide by zero")  
  | _ when x > 1000 -> DivisionSuccessForLargeNumber(x / y)  
  | _ -> DivisionSuccess(x / y)
```

Walkaway

F# has a small, expressive core. Bindings, expressions, type inference, and indentation unlock the rest of the language.

F# Semantics: Session 2

F# language basics

1. The compiler infers types from usage.
2. Records and tuples provide concise data modeling.
3. Understand modules, namespaces, and project organization.

Review

- > How do you return a value in a function?
- > Can you explain this type? `string -> int -> object`
- > How do you change a Record?